API Tutorial

Student: Sam Stoeckel

What It Does

Keeps track of **projects** and the **tasks** inside them.

- Built with PHP and MySQL
- 10 endpoints: 5 for projects, 5 for tasks
- 2 endpoints (the DELETEs) need a Bearer token

Endpoints: Projects

#	Method	Endpoint	What it does	Bearer
1	GET	/projects	List all projects	
2	GET	/projects/{id}	Get one project	
3	POST	/projects	Create a project	
4	PUT	/projects/{id}	Update a project	
5	DELETE	/projects/{id}	Delete a project	*

Endpoints: Tasks

#	Method	Endpoint	What it does	Bearer
6	GET	/projects/{id}/tasks	List a project's tasks	
7	GET	/tasks/{id}	Get one task	
8	POST	/projects/{id}/tasks	Add a task to a project	
9	PUT	/tasks/{id}	Update a task	
10	DELETE	/tasks/{id}	Delete a task	*

Database

projects: `id`, `name`, `description`, `created_at`, `updated_at`

tasks: `id`, `project_id`, `title`, `status`, `due_date`, `created_at`, `updated_at`

- `status` is one of `todo`, `in_progress`, `done`
- `project_id` points to `projects.id`
- Deleting a project also deletes its tasks

Sample data: 2 projects, 3 tasks (`code/schema.sql`)

Authentication

The two DELETE endpoints need this header:

```
Authorization: Bearer admin-demo-token
```

Request	Result
No header	`401` "Bearer token required"
Wrong token	`401` "Invalid or expired token"
Correct token	Request continues

The token is checked **before** anything is deleted.

1. GET /projects

```
curl http://localhost:8080/projects
```

```
{
  "success": true,
  "data": [
    { "id": 1, "name": "Website Redesign", "description": "Refresh the company landing page", ... },
    { "id": 2, "name": "Mobile App", "description": "Build the first version of the iOS app", ... }
  ],
  "count": 2
}
```

2. GET /projects/{id}

```
curl http://localhost:8080/projects/1
```


```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Website Redesign",
    "description": "Refresh the company landing page",
    "created_at": "2026-10-23 21:21:54",
    "updated_at": "2026-10-23 21:21:54"
  }
}
```


An unknown id returns `404` "Project not found".

3. POST /projects

```
curl -X POST http://localhost:8080/projects \
  -H "Content-Type: application/json" \
  -d '{"name":"Marketing Plan","description":"Q1 campaign ideas"}'
```

`201 Created`:

```
{
  "success": true,
  "message": "Project created",
  "data": { "id": 3, "name": "Marketing Plan", "description": "Q1 campaign ideas", ... }
}
```

`name` is required (max 100 characters). `description` is optional.

4. PUT /projects/{id}

```
curl -X PUT http://localhost:8080/projects/3 \
  -H "Content-Type: application/json" \
  -d '{"name":"Marketing Plan 2027","description":"Updated campaign ideas"}'
```

```
{
  "success": true,
  "message": "Project updated",
  "data": { "id": 3, "name": "Marketing Plan 2027", "description": "Updated campaign ideas", ... }
}
```

PUT replaces the whole project, so send every field.

5. DELETE /projects/{id} (Requires Bearer)

```
curl -X DELETE http://localhost:8080/projects/3 \
  -H "Authorization: Bearer admin-demo-token"
```

```
{
  "success": true,
  "message": "Project deleted",
  "data": { "id": 3, "name": "Marketing Plan 2027", ... }
}
```

The project's tasks are deleted with it.

6. GET /projects/{id}/tasks

```
curl http://localhost:8080/projects/1/tasks
```


```
{
  "success": true,
  "data": [
    { "id": 1, "project_id": 1, "title": "Create wireframes", "status": "done", "due_date": "2026-10-01", ... },
    { "id": 2, "project_id": 1, "title": "Write landing page copy", "status": "in_progress", "due_date": "2026-10-15", ... }
  ],
  "count": 2
}
```

7. GET /tasks/{id}

```
curl http://localhost:8080/tasks/2
```


```
{
  "success": true,
  "data": {
    "id": 2,
    "project_id": 1,
    "title": "Write landing page copy",
    "status": "in_progress",
    "due_date": "2026-10-15",
    ...
  }
}
```


An unknown id returns `404` "Task not found".

8. POST /projects/{id}/tasks

```
curl -X POST http://localhost:8080/projects/3/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"Draft social posts","status":"todo","due_date":"2026-11-15"}'
```

`201 Created`:

```
{
  "success": true,
  "message": "Task created",
  "data": { "id": 4, "project_id": 3, "title": "Draft social posts", "status": "todo", "due_date": "2026-11-15", ... }
}
```

`title` is required. `status` defaults to `todo`. `due_date` is optional (`YYYY-MM-DD`).

9. PUT /tasks/{id}

```
curl -X PUT http://localhost:8080/tasks/4 \
  -H "Content-Type: application/json" \
  -d '{"title":"Draft social posts","status":"done","due_date":"2026-11-15"}'
```

```
{
  "success": true,
  "message": "Task updated",
  "data": { "id": 4, "project_id": 3, "title": "Draft social posts", "status": "done", ... }
}
```

10. DELETE /tasks/{id} (Requires Bearer)

```
curl -X DELETE http://localhost:8080/tasks/4 \
  -H "Authorization: Bearer admin-demo-token"
```

```
{
  "success": true,
  "message": "Task deleted",
  "data": { "id": 4, "project_id": 3, "title": "Draft social posts", ... }
}
```

Testing

cURL: `bash code/test_curl.sh`

- 20 checks across all 10 endpoints
- Cleans up the data it creates
- Ends with `Passed: 20  Failed: 0`

Browser: `http://localhost:8080/test_client.html`

- A Send button for every endpoint
- DELETE sections have "With token", "No token", and "Wrong token" buttons

References

ASE 230 Module 1

- *2. Connect PHP Server with MySQL.pdf*, pp. 12–13, 16–17
- *4. PDO Troubleshooting and MySQL (Optional).pdf*, pp. 9–12, 15–19
- *2. Building REST API Server with PHP and JSON.pdf*, pp. 16, 22
- *3. Connecting REST API Server to MySQL with PDO.pdf*, pp. 11–12, 14–22
- *3. Bearer Token Authentication.pdf*, pp. 9–11, 13–14
- *4. Testing REST API Server.pdf*, pp. 8, 15
- Code: `6_RestAPI/api/` , `7_Security/Basics/bearer/`